

Transmission Control Protocol (TCP)

Presentation by:
Dr.N.Sujaudeen
SSN College of Engineering



Introduction

- TCP is a connection-oriented protocol; it creates a virtual connection between two TCPs to send data.
- TCP uses flow and error control mechanisms at the transport level.



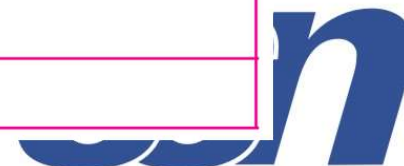
TCP Services

- Process to Process Communication
- Stream Delivery Service
- Sending and Receiving Buffers
- Segments
- Full Duplex communication
- Connection oriented service
- Reliable Service

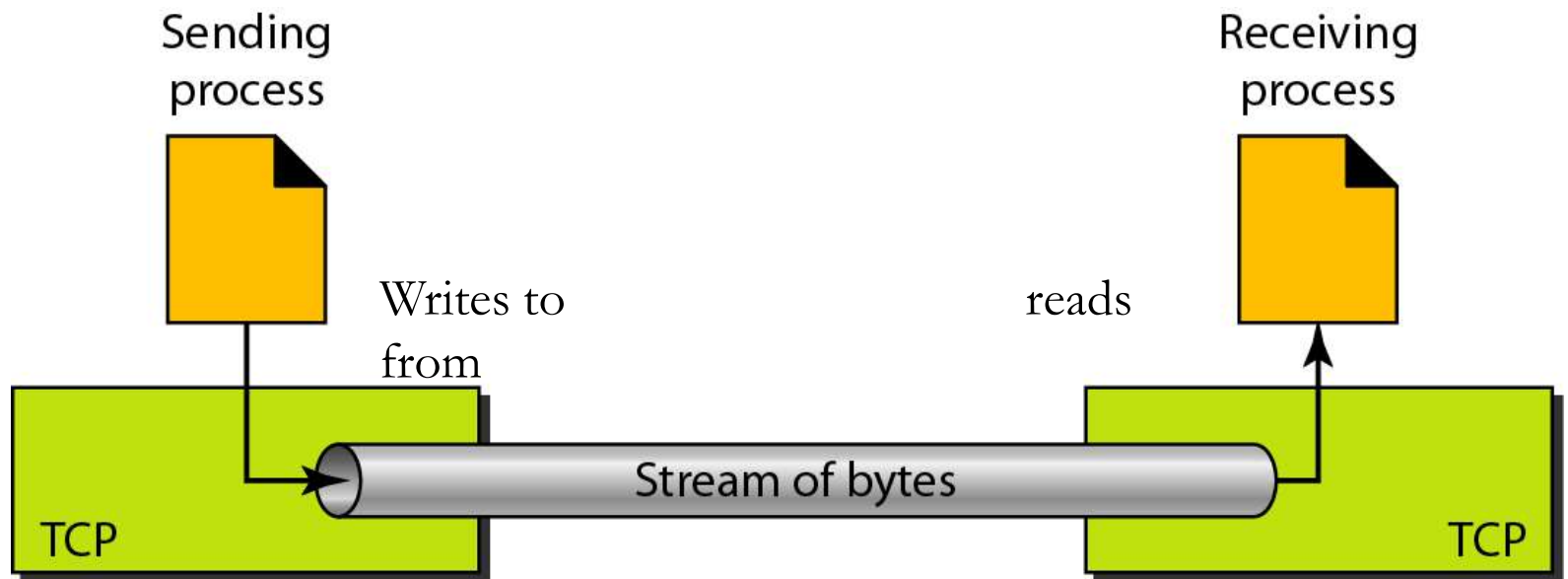


Process to Process Communication

<i>Port</i>	<i>Protocol</i>	<i>Description</i>
7	Echo	Echoes a received datagram back to the sender
9	Discard	Discards any datagram that is received
11	Users	Active users
13	Daytime	Returns the date and the time
17	Quote	Returns a quote of the day
19	Chargen	Returns a string of characters
20	FTP, Data	File Transfer Protocol (data connection)
21	FTP, Control	File Transfer Protocol (control connection)
23	TELNET	Terminal Network
25	SMTP	Simple Mail Transfer Protocol
53	DNS	Domain Name Server
67	BOOTP	Bootstrap Protocol
79	Finger	Finger
80	HTTP	Hypertext Transfer Protocol
111	RPC	Remote Procedure Call

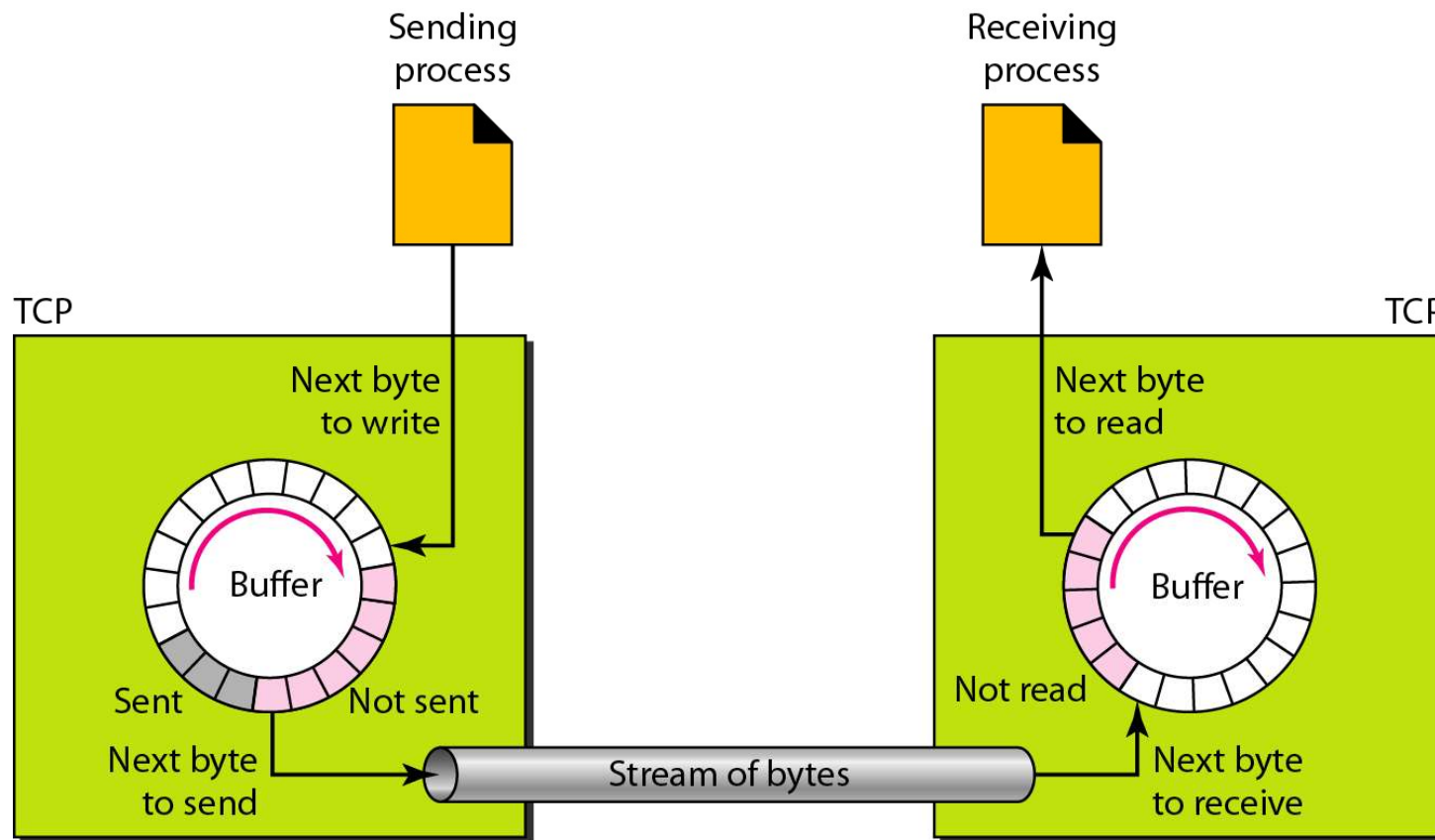


Stream Delivery Service

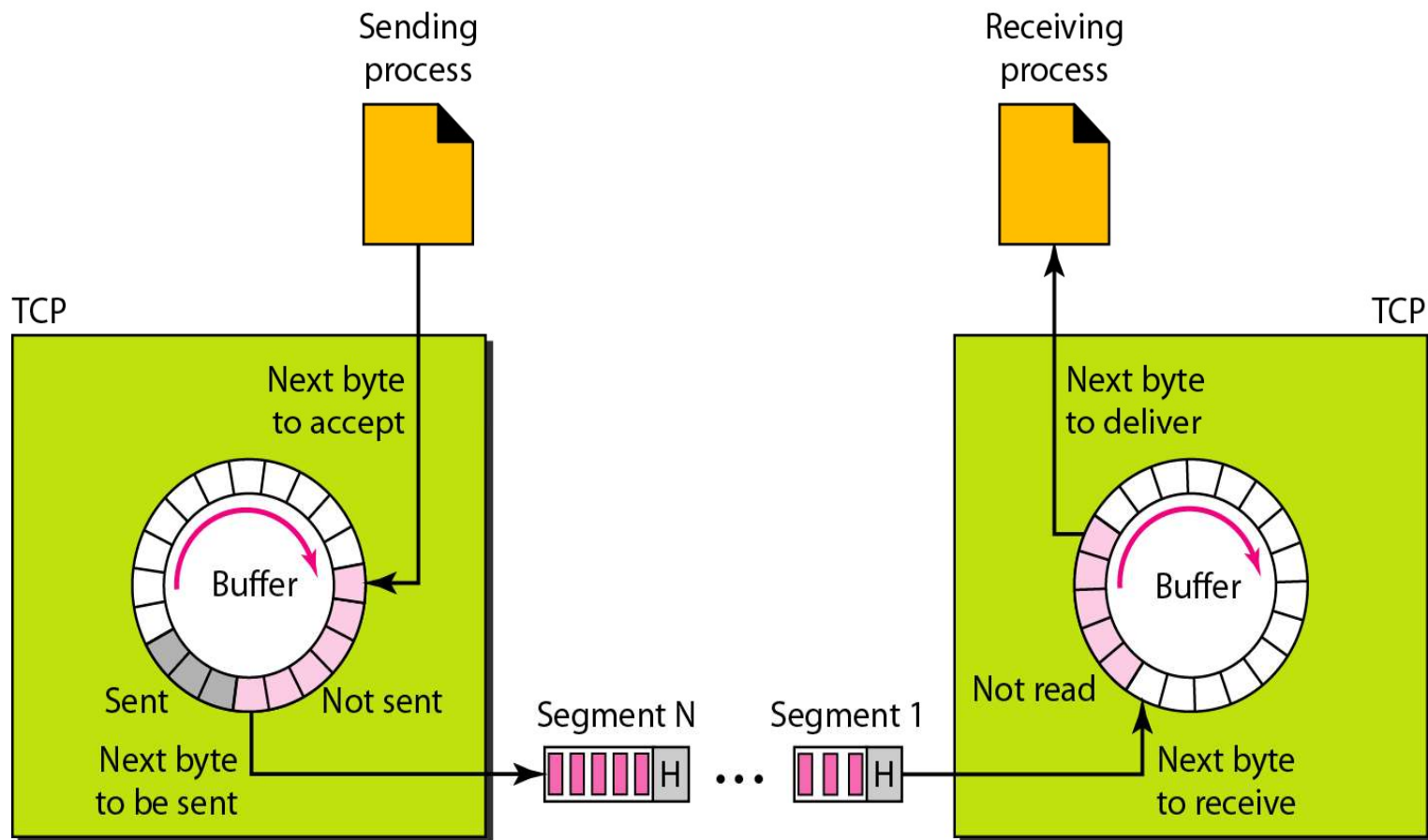


Sending and Receiving Buffers

- Reading and writing are not at the same speed.



Segments



Full Duplex communication

- Data flows in both direction at the same time.
- ie segments can move in both directions at the same time



TCP Features: Numbering System

- Byte Number
- Sequence Number
- Acknowledgment Number



Byte Number

- Each byte should be numbered
- 0 – 6 Random number – 1057
- Data contains 6000 bytes
- 1057 – 7056



Sequence Number

- Sequence number for each segment is the number of the first byte carried in that segment
- Imagine a TCP connection is transferring a file of 6000 bytes. The first byte is numbered 10010. What are the sequence numbers for each segment if data are sent in five segments with the first four segments carrying 1000 bytes and the last segment carrying 2000 bytes?



Sequence Number

- The following shows the sequence number for each segment:

Segment 1 ==> sequence number: 10,010 (range: 10,010 to 11,009)

Segment 2 ==> sequence number: 11,010 (range: 11,010 to 12,009)

Segment 3 ==> sequence number: 12,010 (range: 12,010 to 13,009)

Segment 4 ==> sequence number: 13,010 (range: 13,010 to 14,009)

Segment 5 ==> sequence number: 14,010 (range: 14,010 to 16,009)



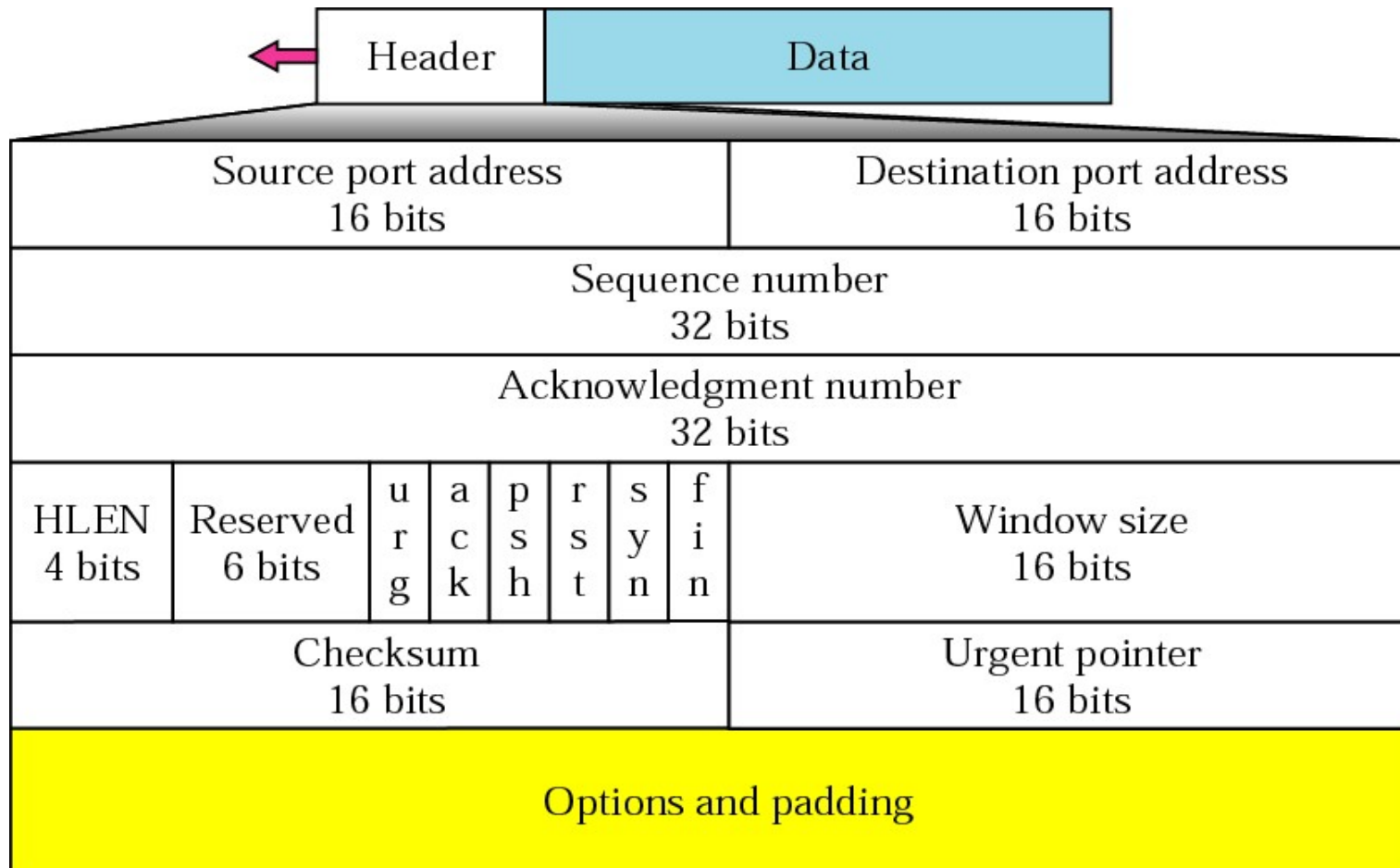
Acknowledgment Number

- The value of the acknowledgment field in a segment defines the number of the next byte a party expects to receive.
- The acknowledgment number is cumulative.



Segment

- 20 or 60 byte header



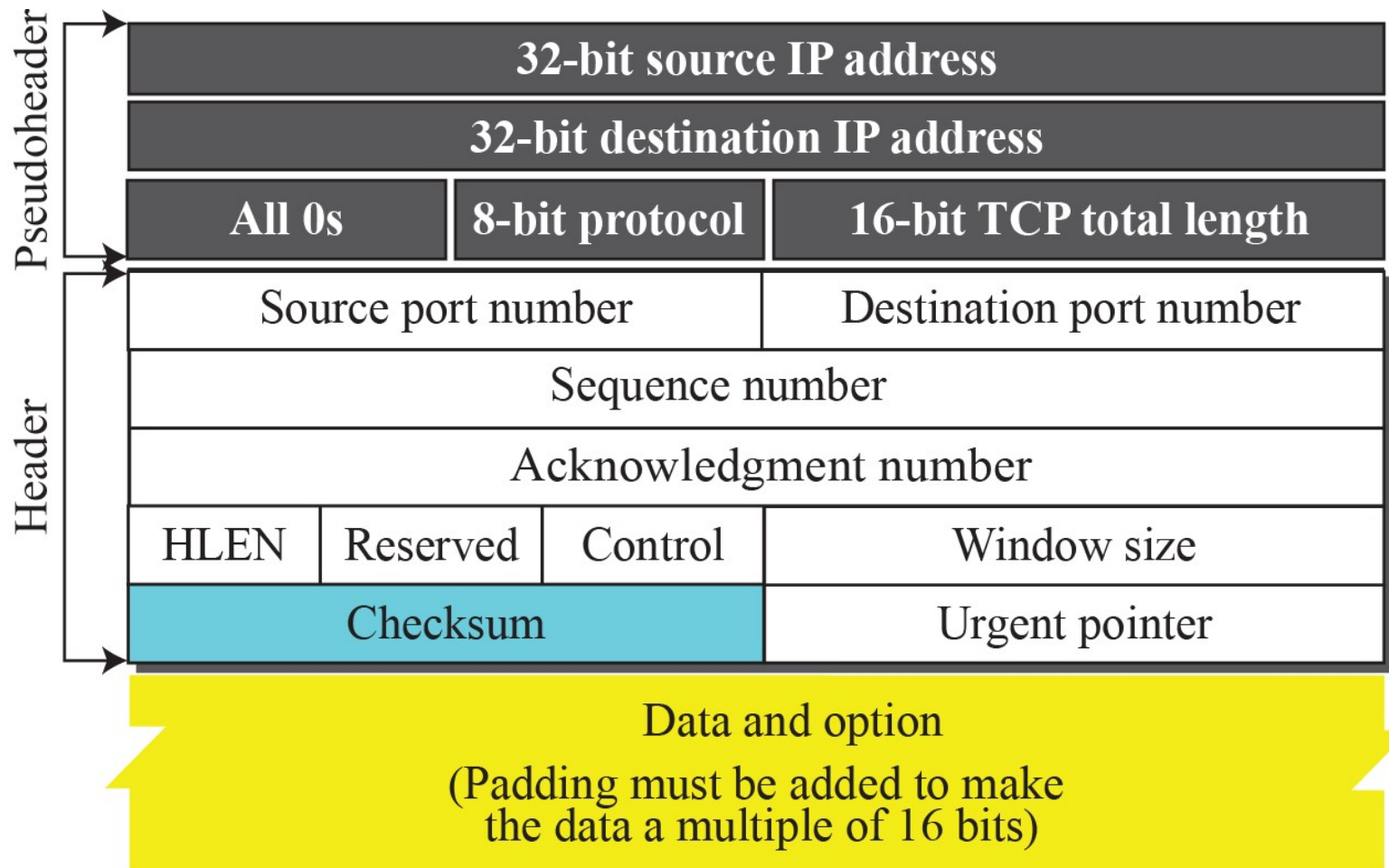
Control Fields

URG: Urgent pointer is valid	RST: Reset the connection
ACK: Acknowledgment is valid	SYN: Synchronize sequence numbers
PSH: Request for push	FIN: Terminate the connection

URG	ACK	PSH	RST	SYN	FIN
-----	-----	-----	-----	-----	-----



Figure 24.9: *Pseudoheader added to the TCP datagram*

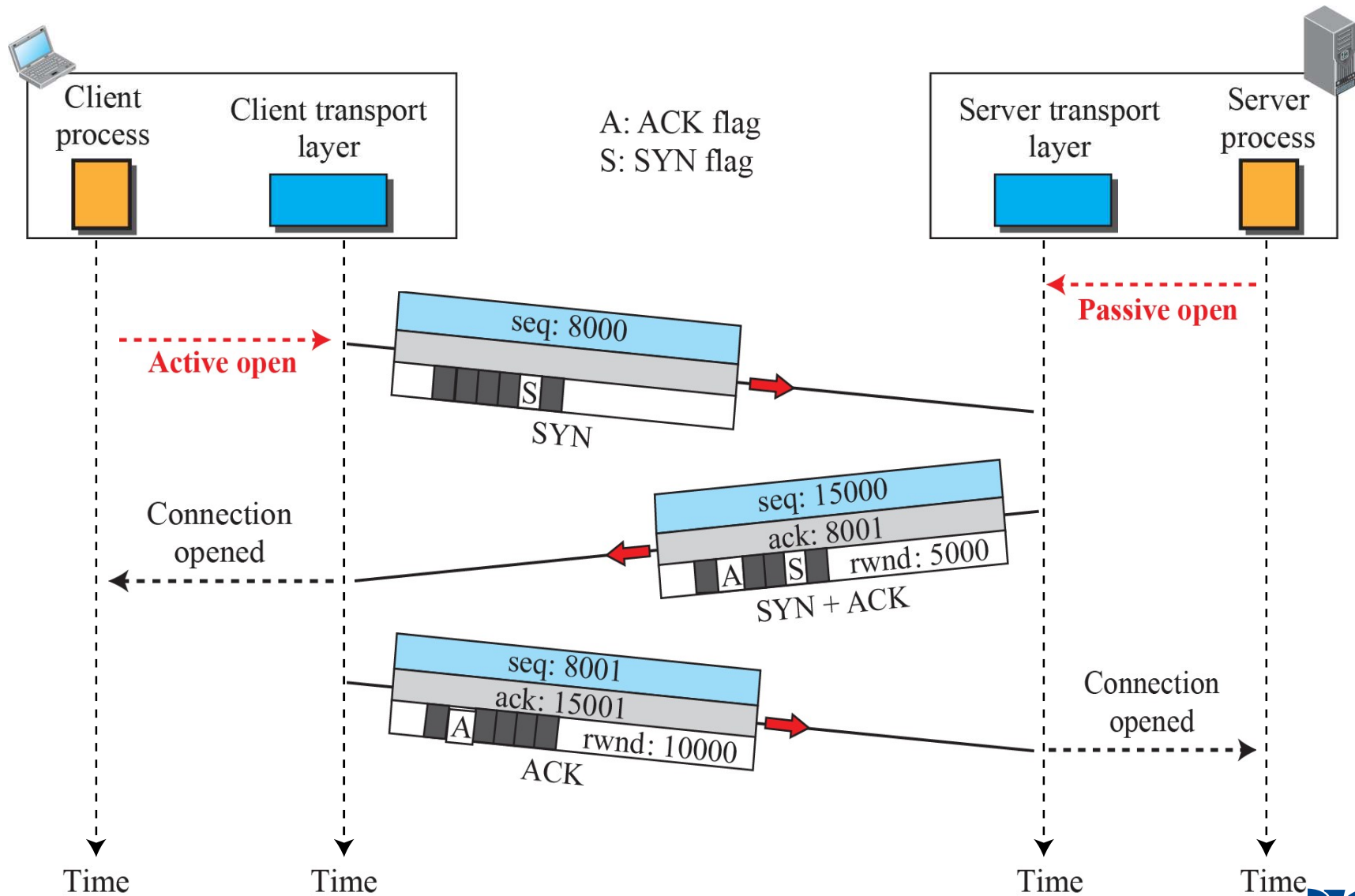




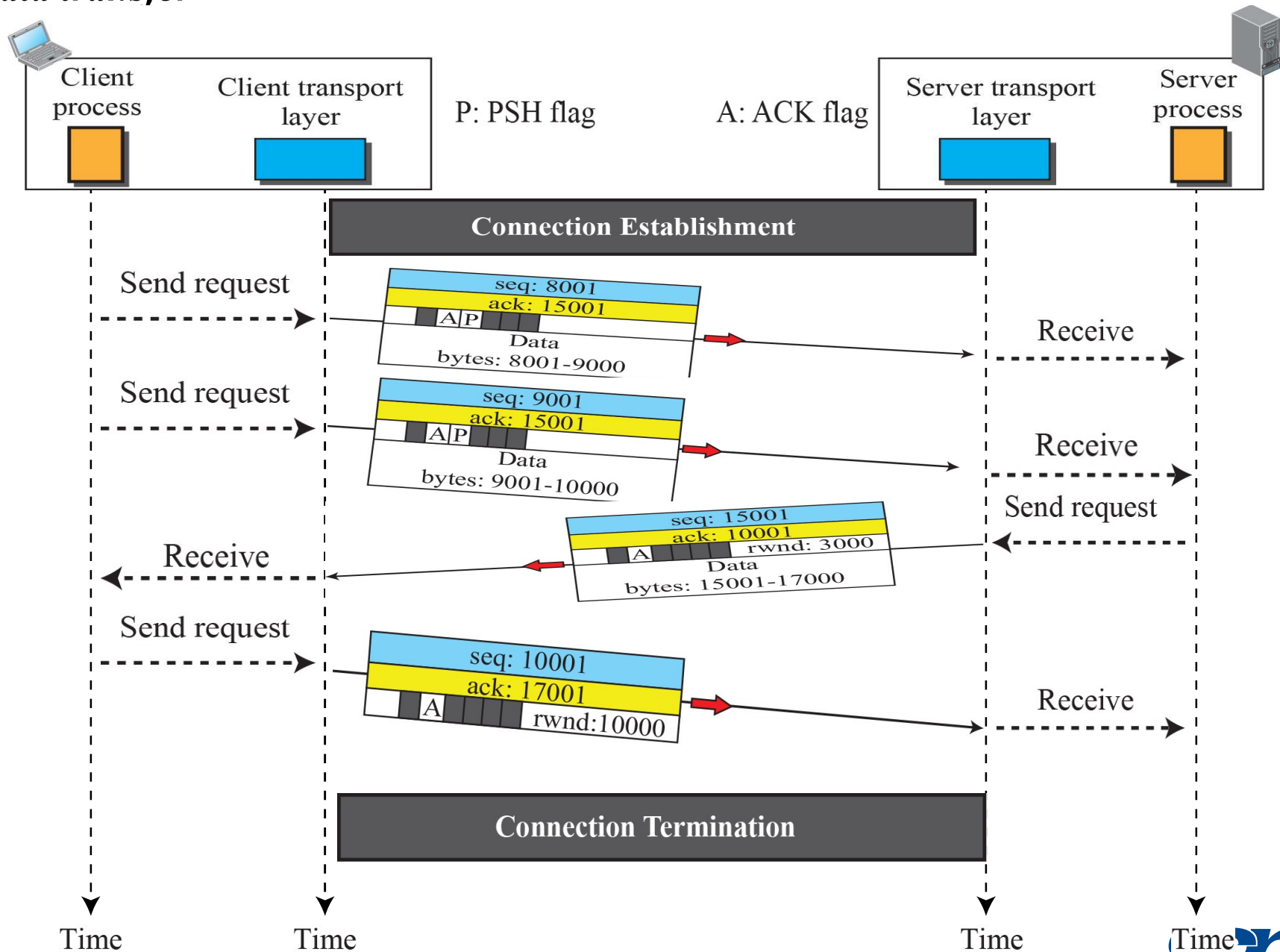
A TCP Connection

TCP is connection-oriented. All of the segments belonging to a message are then sent over this logical path. Using a single logical pathway for the entire message facilitates the acknowledgment process as well as retransmission of damaged or lost frames. You may wonder how TCP, which uses the services of IP, a connectionless protocol, can be connection-oriented. The point is that a TCP connection is logical, not physical. TCP operates at a higher level. TCP uses the services of IP to deliver individual segments to the receiver, but it controls the connection itself.

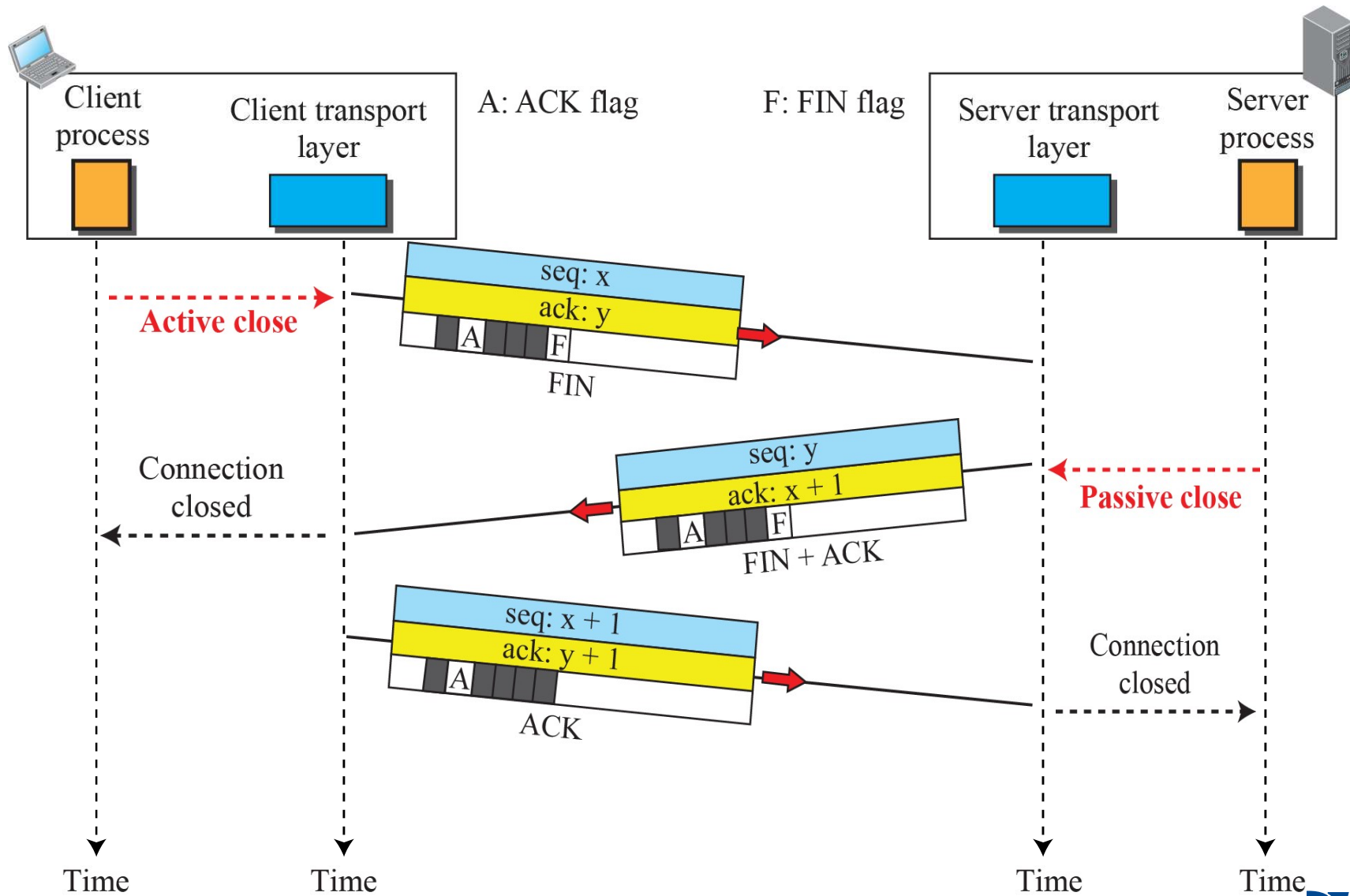
Connection establishment using three-way handshaking



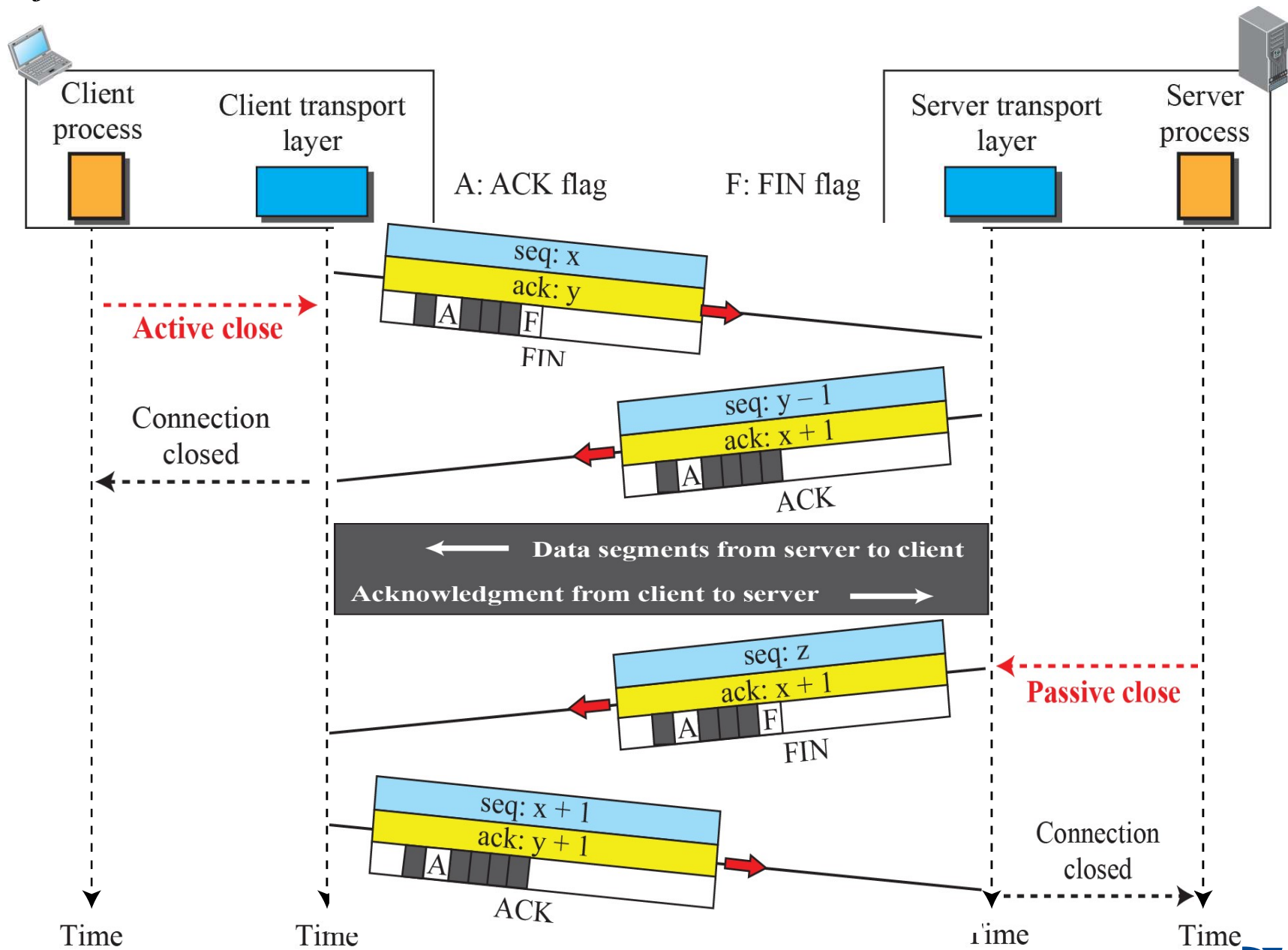
Data transfer



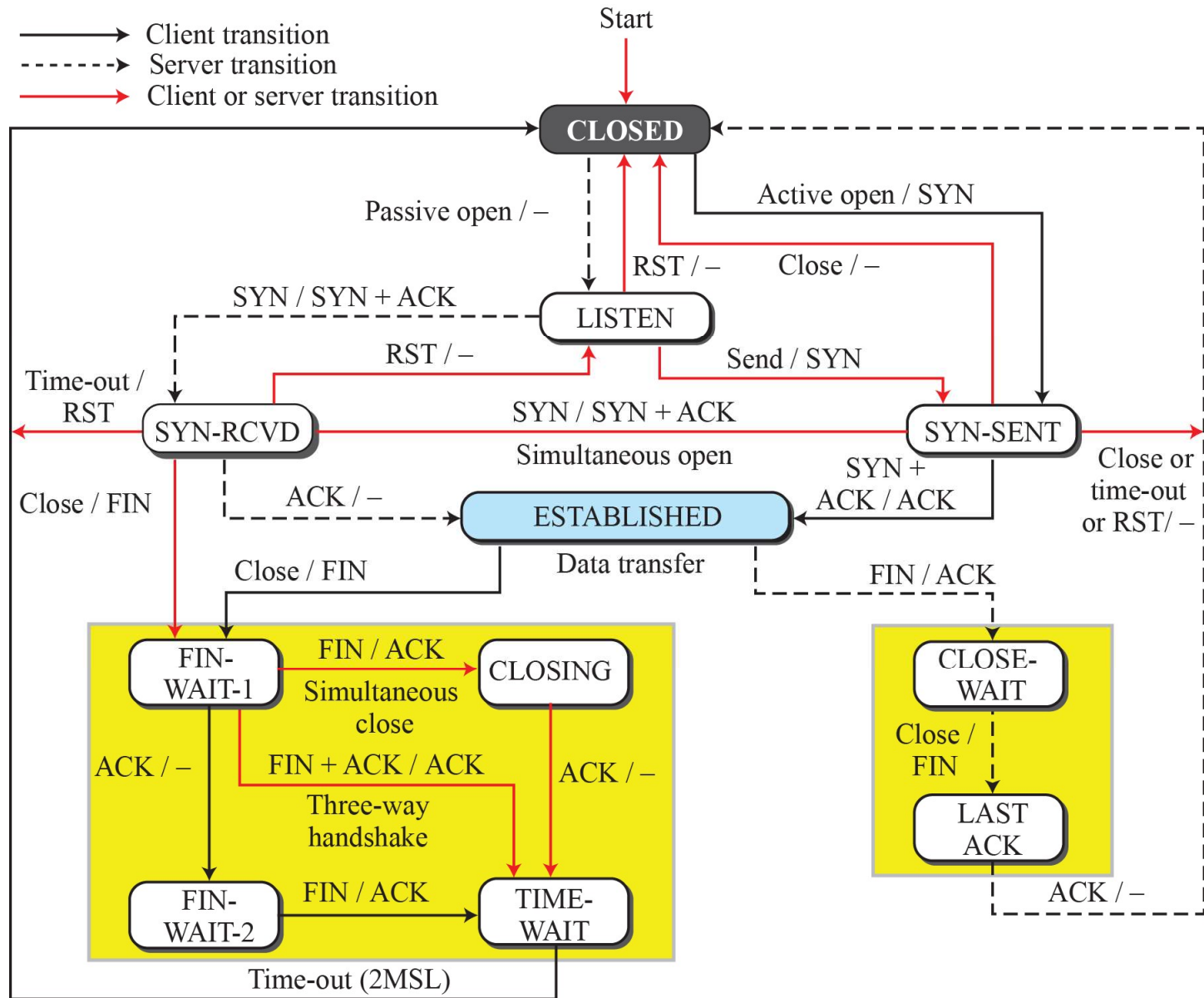
Connection termination using three-way handshaking



Half-close



State Transition Diagram



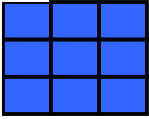
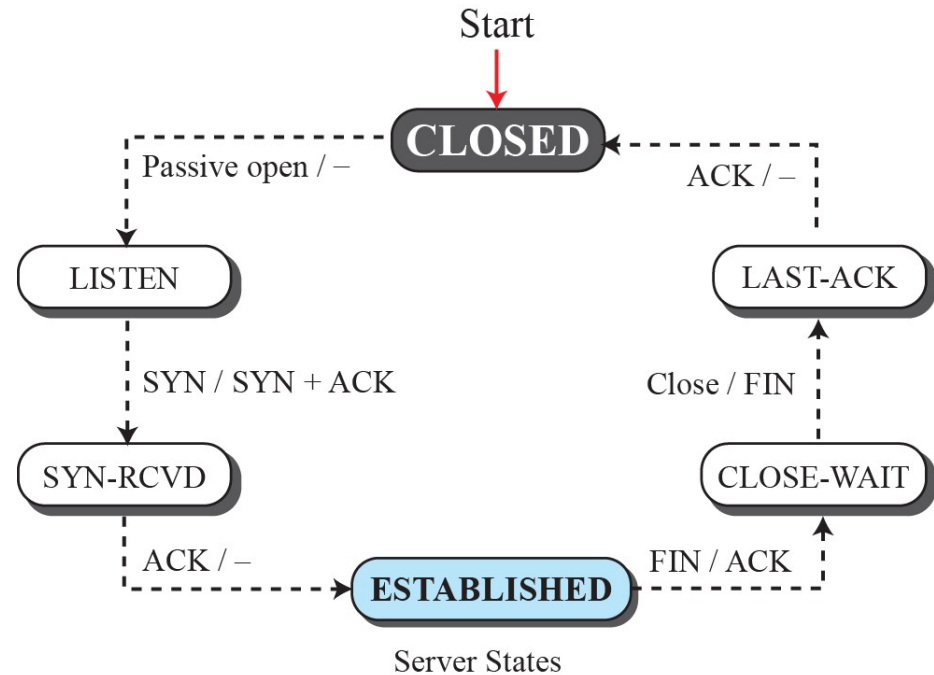
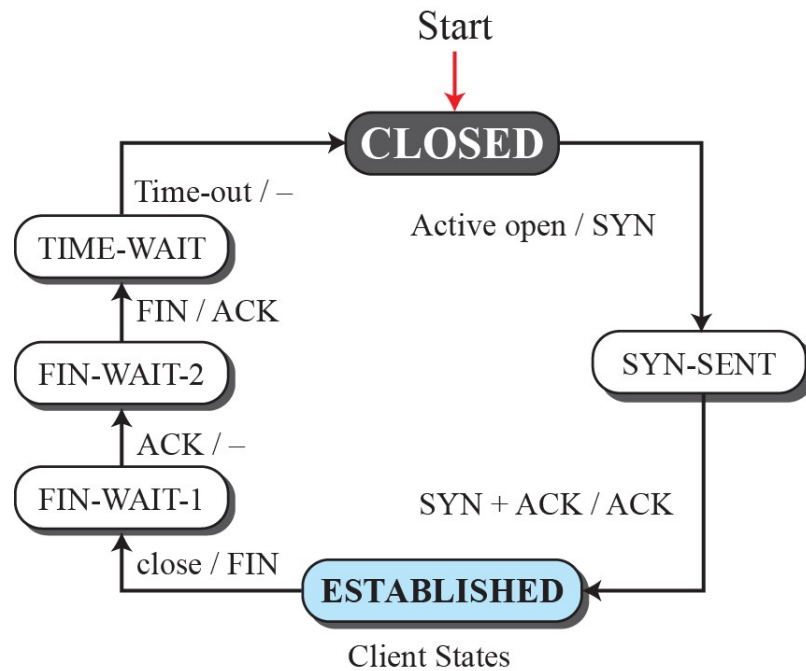


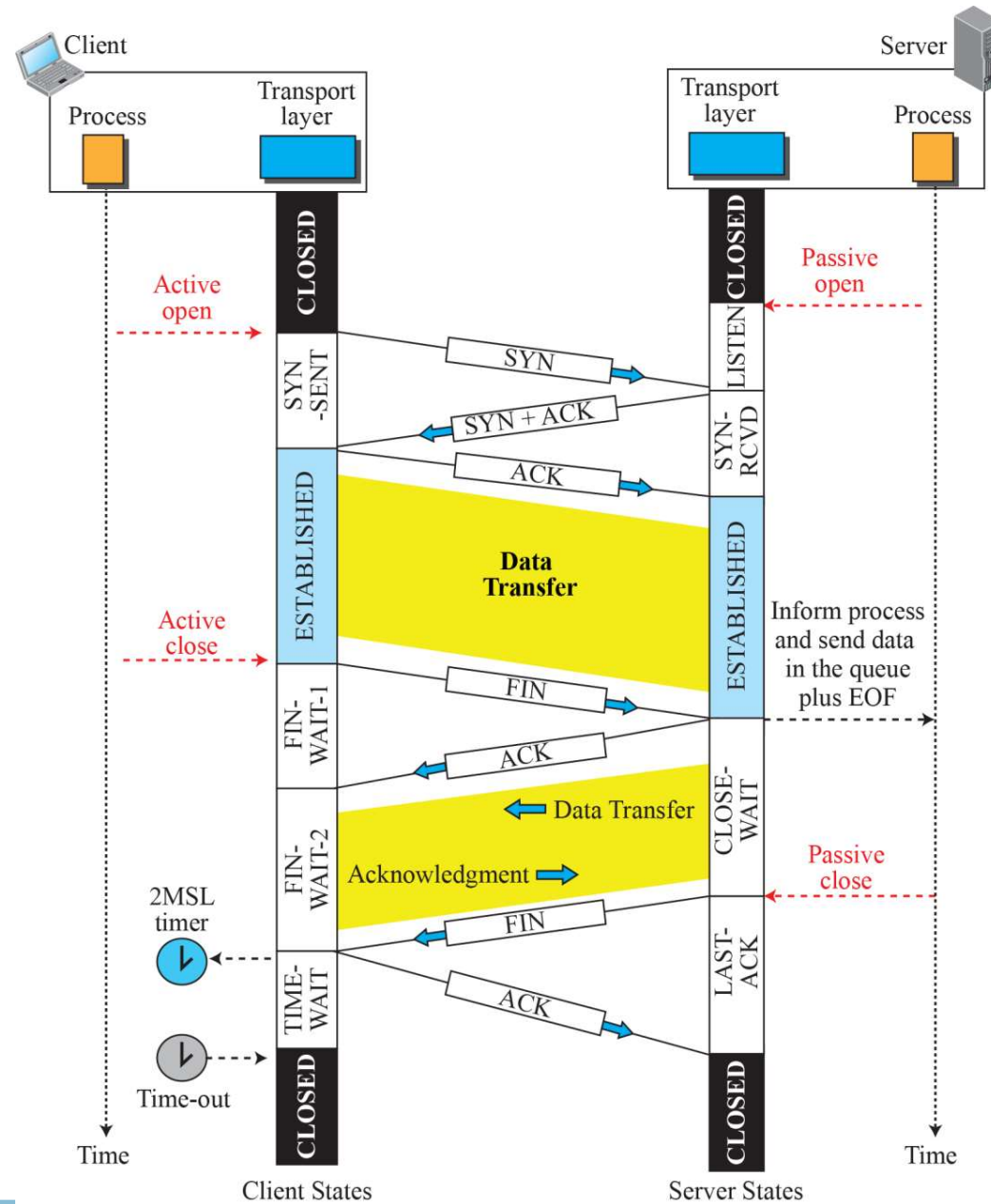
Table 24.2: States for TCP

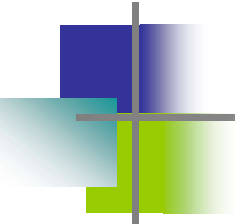
<i>State</i>	<i>Description</i>
CLOSED	No connection exists
LISTEN	Passive open received; waiting for SYN
SYN-SENT	SYN sent; waiting for ACK
SYN-RCVD	SYN+ACK sent; waiting for ACK
ESTABLISHED	Connection established; data transfer in progress
FIN-WAIT-1	First FIN sent; waiting for ACK
FIN-WAIT-2	ACK to first FIN received; waiting for second FIN
CLOSE-WAIT	First FIN received, ACK sent; waiting for application to close
TIME-WAIT	Second FIN received, ACK sent; waiting for 2MSL time-out
LAST-ACK	Second FIN sent; waiting for ACK
CLOSING	Both sides decided to close simultaneously

Transition diagram with half-close connection termination



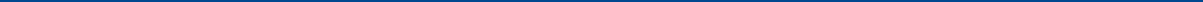
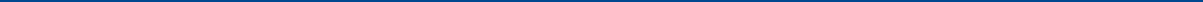
Time-line diagram for a common scenario



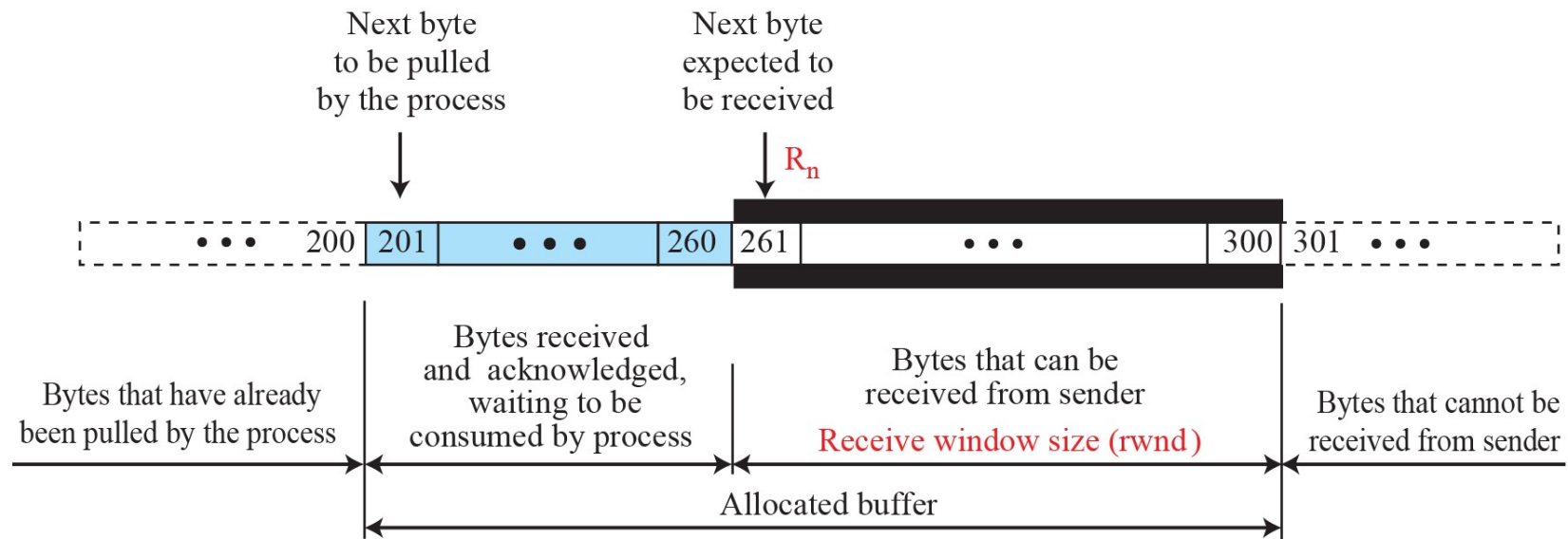


Windows in TCP

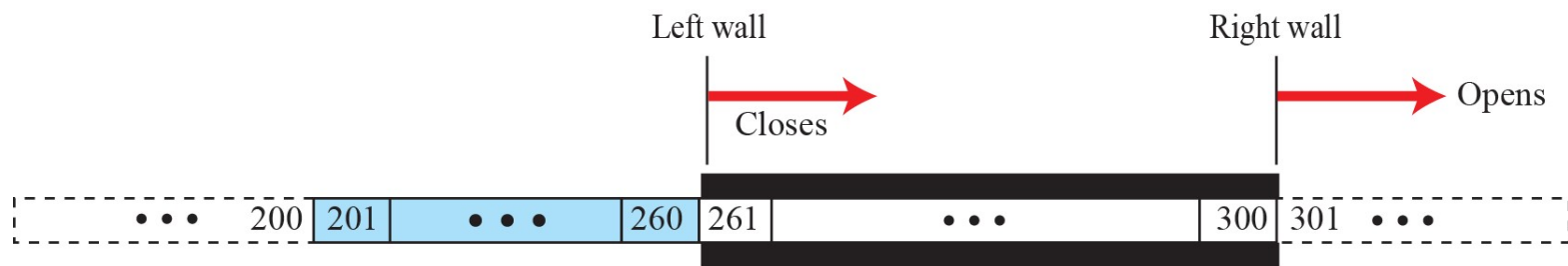
Before discussing data transfer in TCP and the issues such as flow, error, and congestion control, we describe the windows used in TCP. TCP uses two windows (send window and receive window) for each direction of data transfer, which means four windows for a bidirectional communication. To make the discussion simple, we make an unrealistic assumption that communication is only unidirectional. The bidirectional communication can be inferred using two unidirectional communications with piggybacking.



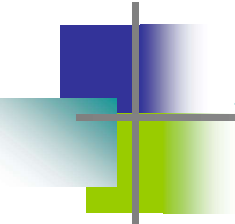
Receive window in TCP



a. Receive window and allocated buffer



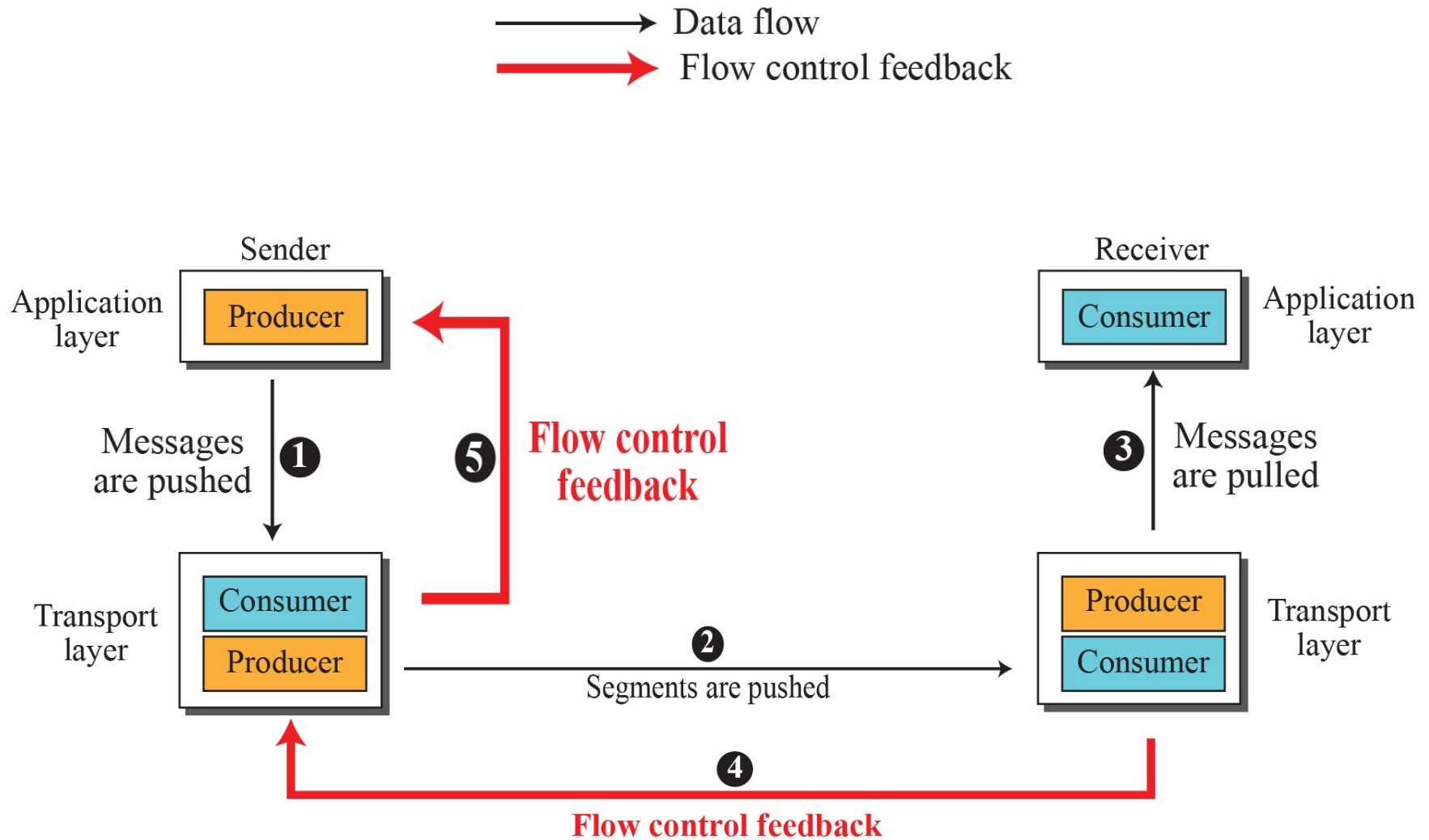
b. Opening and closing of receive window



Flow Control

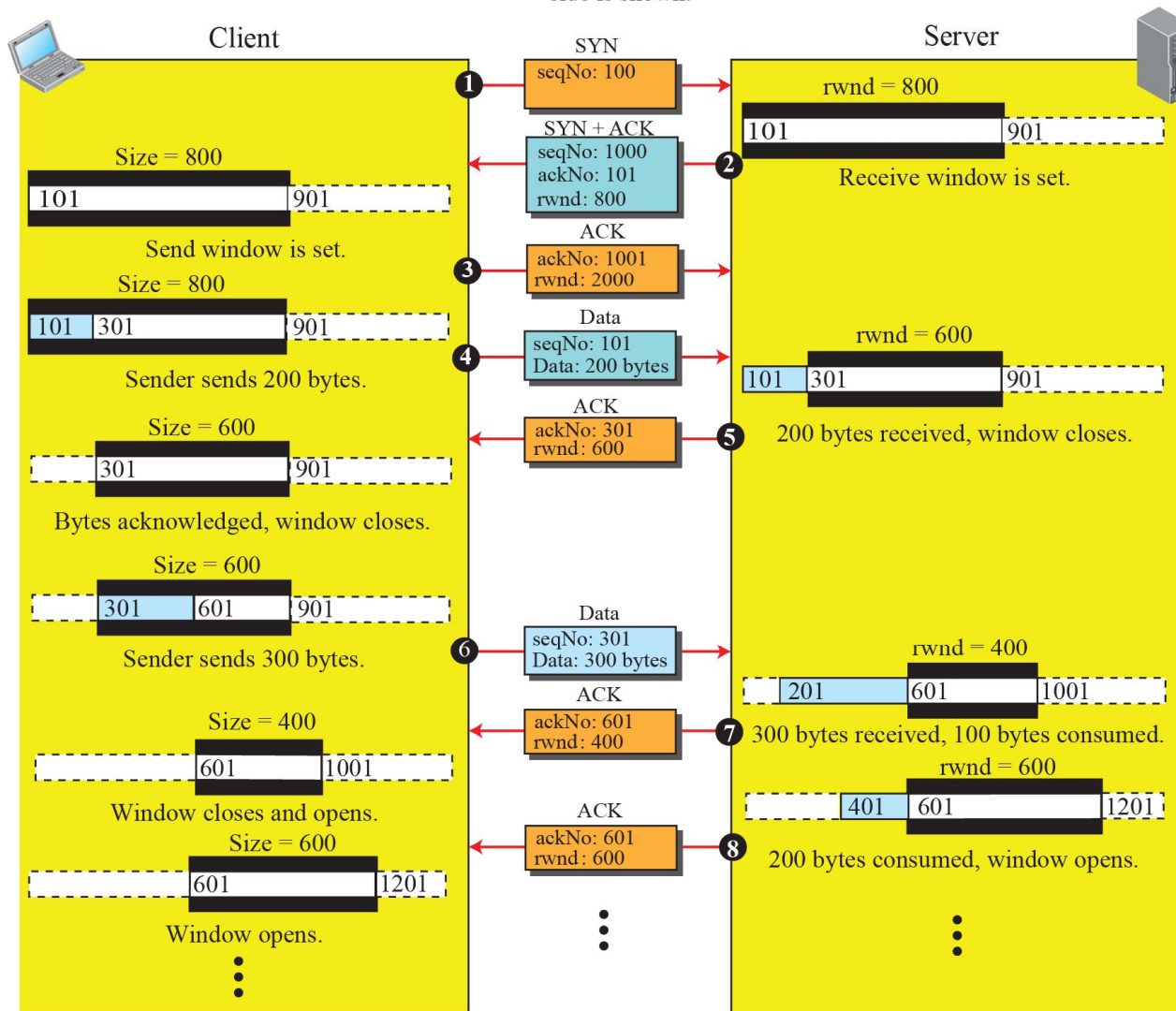
As discussed before, flow control balances the rate a producer creates data with the rate a consumer can use the data. TCP separates flow control from error control. In this section we discuss flow control, ignoring error control. We assume that the logical channel between the sending and receiving TCP is error-free.

Data flow and flow control feedbacks in TCP



An example of flow control

Note: We assume only unidirectional communication from client to server. Therefore, only one window at each side is shown.

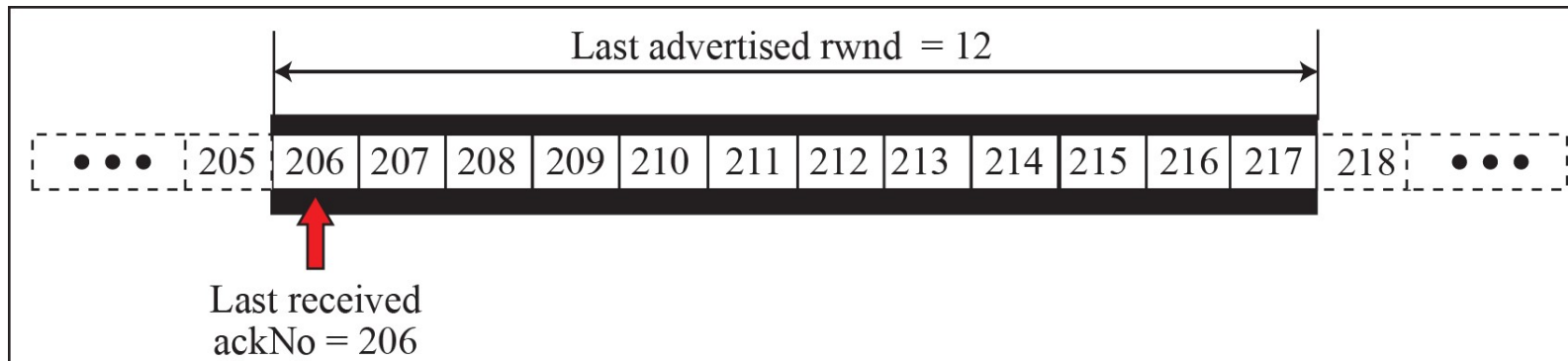


Example

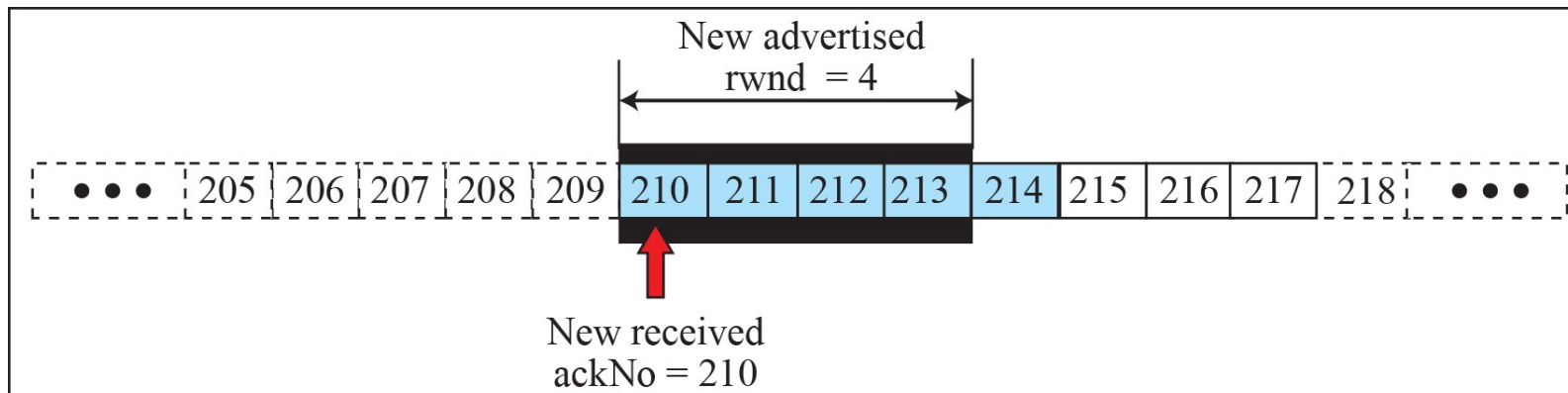
Figure shows the reason for this mandate.

Part a of the figure shows the values of the last acknowledgment and *rwnd*. Part b shows the situation in which the sender has sent bytes 206 to 214. Bytes 206 to 209 are acknowledged and purged. The new advertisement, however, defines the new value of *rwnd* as 4, in which $210 + 4 < 206 + 12$. When the send window shrinks, it creates a problem: byte 214, which has already been sent, is outside the window. The relation discussed before forces the receiver to maintain the right-hand wall of the window to be as shown in part a, because the receiver does not know which of the bytes 210 to 217 has already been sent. described above.

Example



a. The window after the last advertisement



b. The window after the new advertisement; window has shrunk

Thank You...

